

ASSIGNMENT 2

LOGISTIC REGRESSION

QUESTION 1

Implement single layer perceptron for glass classification.

Link for dataset [Click Here](#)

SOLUTION

EDA

```
print(df["Type"].value_counts())
```

```
Type
2    76
1    70
7    29
3    17
5    13
6     9
Name: count, dtype: int64
```

Converting glass_type 1, 2, 3 to “0”

Converting glass_type 5, 6, 7 to “1”

```
df["Window"] = df["Type"].map({1: 0, 2: 0, 3: 0, 4: 0, 5: 1, 6: 1, 7: 1})
```

```
Window
0    163
1     51
Name: count, dtype: int64
```

Selecting Input Features

Possible Columns:

- | | | |
|-------|-------|-------|
| 1. RI | 1. Al | 1. Ca |
| 2. Na | 2. Si | 2. Ba |
| 3. Mg | 3. K | 3. Fe |

Picking the following 4 columns:

```
iv = ["RI", "Mg", "Al", "K"]
X = df[iv].values
y = df["Window"].values
```

Defining the Logistic Regression MLP Class

```
class LogisticRegressionSinglePerceptronModel:
    def __init__(self, eta, inputLayer, outputLayer):
        self.eta = eta
        self.inputLayer = inputLayer
        self.outputLayer = outputLayer
        self.w = np.random.randn(self.inputLayer, self.outputLayer)
        self.delta_E_w = np.zeros((self.inputLayer, self.outputLayer))
        self.E = np.array([])
        self.w_list = []

    iv = ["RI", "Na", "Mg", "Al", "Si", "K", "Ca", "Ba", "Fe"]
    X = df[iv].values

    def sigmoid(self, x):
        return 1 / (1 + np.exp(-x))

    def feedforward(self, X):
        z = np.dot(X, self.w)
        h = self.sigmoid(z)
        return h

    def error(self, h, y):
        error = (-y * np.log(h) - (1 - y) * np.log(1 - h)).mean()
        self.E = np.append(self.E, error)

    def backprop(self, X, y, h):
        self.delta_E_w = np.dot(X.T, h - y) / self.outputLayer
        self.w_list.append(self.w)
        self.w = self.w - eta * self.delta_E_w

    def predict(self, X):
        pred = self.feedforward(X)
        return pred

    def classify(self, y):
        return self.predict(y).round()

    def train(self, X, y):
        for epoch in range(epochs):
            h = self.feedforward(X)
            self.backprop(X, y, h)
            self.error(h, y)
```

Training the Model

```
eta = 0.001
epochs = 10000
inputLayer = X.shape[1]
outputLayer = 1

SPNN = LogisticRegressionSinglePerceptronModel(eta, inputLayer, outputLayer)
SPNN.train(X, y)
pred = SPNN.predict(X)
```

```
pred2 = SPNN.classify(X)
print("Minimum Error achieved:", min(SPNN.E))
```

Output

```
Minimum Error achieved: 0.00015751192266478202
```